Stack Data Structure: From Zero to Interview and Systems Mastery

Elite Data Structures & Algorithms Course

July 14, 2026

Contents

How to Use This Course

A stack is simple to define, but powerful because it models a common rule of work: *the most recent unfinished thing must be handled first.* This course uses the Feynman technique: every idea is explained as if to a beginner, then rebuilt toward expert interview, contest, and real-world engineering use.

For each major topic you will see: definition, intuition, analogy, why it exists, when to use it, when not to use it, internal working, visualization, dry run, pseudocode, code, complexity, edge cases, common mistakes, interview perspective, practice, summary, and mini quiz.

1 Module 0: Why Do Data Structures Exist?

1.1 What Is Data?

Big idea. Data is information stored in a form a computer can process. A name, a number, a paragraph, an image pixel, a bank balance, and a browser tab are all data.

A computer does not understand meaning directly. It stores bits: zeros and ones. Programs give those bits structure.

```
Human idea: age = 21
Program variable: int age = 21
Memory view:
+-------------+-------------+
| Address     | Value       |
+-------------+-------------+
| 0x1000      | 21          |
+-------------+-------------+
```

1.2 Why Can't We Simply Use Variables?

A variable is like one labeled box. It works when you have one thing.

```
score = 95
```

But real software rarely stores just one thing. It stores thousands or billions of related things: messages in a chat, search results, undo history, open function calls, game states, orders in a marketplace.

If you tried to use separate variables, the program would become impossible to manage:

```
message1, message2, message3, ..., message1000000
```

Data structures exist because programmers need disciplined ways to store, organize, access, modify, and reason about many pieces of data.

1.3 Why Are Arrays Not Enough?

An array stores items side by side in memory and lets us access an item by index.

```
Array A = [10, 20, 30, 40]
Index     0   1   2   3
Memory: +----+----+----+----+
        | 10 | 20 | 30 | 40 |
        +----+----+----+----+
```

Arrays are excellent when you need direct indexed access. But arrays do not by themselves express rules such as:

- "Only remove the most recently added item."
- "Always process the oldest waiting customer first."

- "Find the smallest item quickly."
- "Connect cities and roads."

Those rules need specialized structures: stacks, queues, heaps, trees, graphs, hash tables, and more.

1.4 Historical Context and Evolution

Early programmers worked close to hardware. They had memory addresses and sequences of instructions. As programs grew, they repeatedly faced the same storage problems: temporary calculations, nested function calls, expression parsing, scheduling, searching, and file organization.

Data structures evolved as reusable answers:

Problem	Structure that helps
Many items in order	Array, linked list
Most recent item first	Stack
Oldest item first	Queue
Fast lookup by key	Hash table
Sorted hierarchical data	Binary search tree
Network relationships	Graph
Priority-based processing	Heap / priority queue

1.5 Where Stack Fits

A stack is a restricted list. It does not try to do everything. It intentionally allows access at one end only, called the top. This restriction is its strength: because the rule is simple, operations are fast and reasoning is clean.

```
General list: insert/remove anywhere
Queue:        insert at back, remove from front
Stack:        insert at top, remove from top
```

Big idea. Stack is the structure for nested, reversible, most-recent-first work.

Checkpoint: master this before continuing

- Summary: variables store single values; arrays store indexed sequences; data structures encode access rules; stacks encode most-recent-first access.
- Concept questions: What is data? Why are many separate variables bad? What rule does a stack enforce? Why can restriction be useful? Name a real software stack.

- MCQ answers: LIFO means last-in-first-out; arrays are contiguous; stack top is the accessible end; browser back uses a stack-like history; graph is for relationships.
- Dry runs: Draw memory for one variable, an array of five values, and a stack of five values.
- Coding warm-up: Store three numbers in variables, then in an array, then imagine only reading the last inserted one.

2 Module 1: Introduction to Stack

2.1 Intuition Before Definition

Imagine a pile of plates. You put a clean plate on top. When someone needs a plate, they take the top plate. Taking a plate from the middle is possible physically, but awkward and dangerous. The natural rule is: last plate placed is first plate removed.

```
Top
 |
 v
+--------+
| Plate4 | <- last added, first removed
+--------+
| Plate3 |
+--------+
| Plate2 |
+--------+
| Plate1 |
+--------+
```

2.2 Definition

A stack is a linear data structure in which insertion and deletion happen at the same end, called the top. It follows LIFO: Last In, First Out.

2.3 Why Was Stack Invented?

Stacks solve problems where the newest pending item must be resolved before older items. This appears naturally in:

- function calls: the current function must finish before its caller continues;
- undo: the last edit should be undone first;
- browser back: the current page returns to the previous page;
- expression parsing: the most recent unmatched symbol matters;
- backtracking: the latest decision is reversed first.

2.4 Why LIFO, Not FIFO?

FIFO means first-in-first-out, like a queue at a ticket counter. FIFO is fair for waiting customers. LIFO is correct for nested work.

```
Nested task example:
Open A
  Open B
    Open C
```

```
    Close C first
  Close B next
Close A last
```

You cannot close A before C if C is inside B and B is inside A. This nested structure is exactly stack-shaped.

2.5 Operation Visualization

```
Start: empty

push(10)
Top -> +----+
       | 10 |
       +----+

push(20)
Top -> +----+
       | 20 |
       +----+
       | 10 |
       +----+

pop() returns 20
Top -> +----+
       | 10 |
       +----+
```

Checkpoint: master this before continuing

- Summary: a stack stores items so the last inserted item is the first removed.
- Concept questions: Why is undo LIFO? Why is a printer queue usually not LIFO? What is the top? Can you access the bottom directly? Why do nested tasks match stacks?
- Practice: Draw stacks after push(5), push(7), pop(), push(9), peek(). Answer: top is 9 and stack bottom-to-top is 5, 9.
- Interview question: "When would a stack be wrong?" Answer: when the oldest item must be processed first, such as customer service lines.

3 Module 2: Stack Properties

3.1 Core Properties

Property	Meaning
LIFO	Last inserted item is removed first.
Top pointer	Tracks the currently accessible item.
Push	Insert item at top.
Pop	Remove and return top item.
Peek / top	Return top item without removing it.
isEmpty	Check whether no items exist.
isFull	Check whether fixed-capacity storage is full.
Size	Number of current items.
Capacity	Maximum number of items a static stack can hold.
Static stack	Fixed capacity, usually array-backed.
Dynamic stack	Can grow, commonly via resizing array or linked list.

3.2 Why Top Starts at -1 in Array Stacks

If an array index starts at 0, an empty stack has no valid top index. We use -1 to mean "no item."

```
capacity = 5
array: [ _, _, _, _, _ ]
top = -1  means empty

push(10): top becomes 0, array[0] = 10
array: [10, _, _, _, _]
top = 0
```

3.3 Static vs Dynamic Stack

A static stack is like a parking lot with fixed spaces. It is fast and predictable, but can overflow. A dynamic stack is like a notebook where you can add pages; it is flexible, but may need memory allocation.

Checkpoint: master this before continuing

- Summary: stack behavior is controlled by the top; static stacks can become full; dynamic stacks can grow.
- Concept questions: What is capacity? What is size? Why is top -1 when empty? What is overflow? What is underflow?
- Dry run: capacity 3, push A, push B, push C, push D. Answer: push D causes overflow in a static stack.

4 Module 3: Stack Operations

4.1 Operation Algorithms

Operation	Logic	Complexity
push(x)	If not full, move top upward and store x.	**Time:** $O(1)$ **Extra space:** $O(1)$
pop()	If not empty, return top item and move top downward.	**Time:** $O(1)$ **Extra space:** $O(1)$
peek()	If not empty, read top item only.	**Time:** $O(1)$ **Extra space:** $O(1)$
display()	Visit items from top to bottom or bottom to top.	**Time:** $O(n)$ **Extra space:** $O(1)$
size()	Return stored count or top+1.	**Time:** $O(1)$ **Extra space:** $O(1)$
isEmpty()	Test top == -1 or head == null.	**Time:** $O(1)$ **Extra space:** $O(1)$
isFull()	Test top == capacity - 1 for static arrays.	**Time:** $O(1)$ **Extra space:** $O(1)$
clear()	Reset top or head.	**Time:** $O(1)$ or $O(n)$ **Extra space:** $O(1)$
search(x)	Scan items until x is found.	**Time:** $O(n)$ **Extra space:** $O(1)$
reverse()	Move items through helper stacks or reverse links.	**Time:** $O(n)$ **Extra space:** $O(n)$ or $O(1)$
copy/clone()	Duplicate each item preserving order.	**Time:** $O(n)$ **Extra space:** $O(n)$
merge()	Combine stacks using a chosen order rule.	**Time:** $O(n+m)$ **Extra space:** $O(n+m)$

4.2 Pseudocode

```
PUSH(stack, x):
    if stack is full:
        report overflow
    else:
        top = top + 1
        array[top] = x

POP(stack):
    if stack is empty:
        report underflow
    else:
        x = array[top]
        top = top - 1
        return x

PEEK(stack):
    if stack is empty:
```

```
        report underflow
    else:
        return array[top]
```

4.3 Dry Run

```
capacity = 4, top = -1
push(10): top=0, [10, _, _, _]
push(20): top=1, [10,20, _, _]
peek(): returns 20, top=1
pop(): returns 20, top=0, [10,20, _, _] but 20 is ignored
push(30): top=1, [10,30, _, _]
```

Notice pop does not need to erase the old memory cell in many languages. It only changes which part of the array is considered active.

4.4 Common Mistakes

- Forgetting underflow: popping from an empty stack.
- Forgetting overflow in fixed arrays.
- Returning array[top] after decrementing top instead of before.
- Confusing display order with logical stack order.
- Assuming search is $O(1)$; it is $O(n)$ because only the top has direct stack access.

Practice set with answers

1. Concept: Why does pop read before decrement? Answer: after decrement, top points to the next item, not the removed item.
2. MCQ: push is usually A) $O(1)$ B) $O(n)$ C) $O(\log n)$. Answer: A.
3. Dry run: push 1, push 2, push 3, pop, pop, push 4. Answer: bottom-to-top is 1, 4.
4. Coding: implement isEmpty as top == -1.
5. Interview: What are stack underflow and overflow? Answer: invalid removal from empty stack and invalid insertion into full fixed stack.

5 Module 4: Stack Implementation Using Array

5.1 How Arrays Work Internally

An array is a contiguous block of memory. If each integer uses 4 bytes and the first element starts at address 1000, then index 3 is at address $1000 + 3 \times 4 = 1012$.

```
Index:   0     1     2     3
Value:  10    20    30    40
Addr: 1000  1004  1008  1012
```

This explains why array indexing is fast: the CPU computes an address directly.

5.2 Array Stack Code in Python

```
class ArrayStack:
    def __init__(self, capacity):
        self.capacity = capacity
        self.data = [None] * capacity
        self.top = -1

    def is_empty(self):
        return self.top == -1

    def is_full(self):
        return self.top == self.capacity - 1

    def push(self, value):
        if self.is_full():
            raise OverflowError("stack overflow")
        self.top += 1
        self.data[self.top] = value

    def pop(self):
        if self.is_empty():
            raise IndexError("stack underflow")
        value = self.data[self.top]
        self.data[self.top] = None
        self.top -= 1
        return value

    def peek(self):
        if self.is_empty():
            raise IndexError("empty stack")
        return self.data[self.top]

    def size(self):
        return self.top + 1

    def display(self):
        for i in range(self.top, -1, -1):
            print(self.data[i])
```

5.3 Line-by-Line Explanation

- `capacity`: the maximum number of cells reserved.
- `data`: the array holding values.
- `top = -1`: no valid item exists yet.
- `push`: checks space, increments top, stores value.
- `pop`: checks non-empty, saves top value, clears cell, decrements top.
- `peek`: reads without changing top.

5.4 Why Increment First and Decrement Later?

For push, the next free cell is one above current top, so increment first. For pop, current top is the item to return, so read first and decrement later.

5.5 Advantages and Limitations

Advantages	Disadvantages
Simple, cache-friendly, fast indexing	Fixed capacity unless resized
Low per-item memory overhead	Overflow possible in static version
Great for competitive programming	Insertion not allowed except at top

Checkpoint: master this before continuing

- Summary: an array stack uses a top index to treat an array as a LIFO structure.
- Interview questions: Why top = -1? What is overflow? Why is array stack cache-friendly? What is amortized push in a resizing array?

6 Module 5: Stack Implementation Using Linked List

6.1 Nodes, References, and Heap Memory

A linked list is built from nodes. A node stores data and a reference to another node.

```
+------+------+
| data | next |
+------+------+
```

Unlike arrays, nodes do not need to be side by side in memory.

```
head
 |
 v
+----+----+     +----+----+     +----+------+
| 30 |  *------>| 20 |  *------>| 10 | null |
+----+----+     +----+----+     +----+------+
```

For a stack, the head of the linked list is the top. This makes push and pop $O(1)$.

6.2 Linked List Stack Code in Python

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

class LinkedStack:
    def __init__(self):
        self.head = None
        self.count = 0

    def is_empty(self):
        return self.head is None

    def push(self, value):
        node = Node(value)
        node.next = self.head
        self.head = node
        self.count += 1

    def pop(self):
        if self.is_empty():
            raise IndexError("stack underflow")
        value = self.head.value
        self.head = self.head.next
        self.count -= 1
        return value

    def peek(self):
        if self.is_empty():
            raise IndexError("empty stack")
```

```
        return self.head.value

    def size(self):
        return self.count
```

6.3 Internal Push Dry Run

```
Before push(40):
head -> [30|*] -> [20|*] -> [10|null]

Create node [40|null]
node.next = head
head = node

After:
head -> [40|*] -> [30|*] -> [20|*] -> [10|null]
```

6.4 Memory Deallocation and Garbage Collection

In C/C++, removed nodes should be explicitly freed or deleted. In Java, Python, C#, Go, and JavaScript, unreachable nodes are eventually reclaimed by garbage collection.

6.5 Advantages and Disadvantages

- Advantage: grows dynamically until memory is exhausted.
- Advantage: no fixed capacity overflow.
- Disadvantage: each node stores an extra reference/pointer.
- Disadvantage: worse cache locality than arrays because nodes may be scattered.

7 Module 6: Array vs Linked List Stack

Aspect	Array Stack	Linked List Stack
Memory layout	Contiguous	Scattered nodes
Push/pop	$O(1)$, resizing may be amortized	Always $O(1)$ if allocation succeeds
Cache locality	Excellent	Poorer
Overflow	Possible if fixed capacity	No fixed overflow
Memory waste	Unused capacity may exist	Pointer overhead per node
Competitive programming	Often preferred for speed	Used when size unknown
Production	Dynamic arrays often common	Useful for frequent structural changes
Interview	Easy to implement	Tests pointer understanding

When to use. Use an array stack when you know a good capacity or want speed and cache efficiency. Use a linked stack when size is highly unpredictable and avoiding resizing is important. **When not to use.** Do not use a stack when you need random access, sorted retrieval, or oldest-first service.

8 Module 7: Internal Working Inside the Computer

8.1 CPU, RAM, and Addresses

The CPU executes instructions. RAM stores program data. A variable or array cell lives at some address. Pointers and references are values that identify where another object is stored.

8.2 During Array Push

```
Instruction idea:
1. Compare top with capacity - 1.
2. Add 1 to top.
3. Compute address: base + top * element_size.
4. Store value at that address.
```

8.3 During Linked Push

```
1. Ask heap for memory for a new node.
2. Store value in node.value.
3. Store old head address in node.next.
4. Store node address in head.
```

8.4 Call Stack Clarification

The term "stack" also appears in runtime memory. When a function calls another function, the runtime stores a stack frame containing return address, parameters, and local variables.

```
main calls A, A calls B, B calls C

Top -> C frame
       B frame
       A frame
       main frame
```

C must finish before B continues. That is LIFO.

9 Module 8: Time and Space Complexity

9.1 Big-O From Basics

Big-O describes how work grows as input size grows. It ignores machine-specific constants and focuses on growth trend.

- $O(1)$: constant work. Reading the top item takes one direct action.
- $O(n)$: linear work. Displaying all items touches every item.
- $O(\log n)$: work grows by repeated halving, not typical for basic stack operations.

9.2 Deriving Stack Operation Complexity

Push does not inspect all items. It only checks a condition, updates one pointer/index, and writes one item. Therefore it is $O(1)$. Pop similarly reads one item and updates one pointer/index. Display and search may inspect every item, so they are $O(n)$.

9.3 Amortized Complexity

A dynamic array stack may occasionally resize by copying all n items. One resize is $O(n)$, but if capacity doubles, many cheap pushes occur between expensive copies. Spread over many operations, push is amortized $O(1)$.

10 Module 9: Stack Applications

10.1 Function Call Stack and Recursion

Recursion means a function calls itself. Each call waits for a deeper call to finish.

```
def factorial(n):
    if n == 0:
        return 1
    return n * factorial(n - 1)
```

```
factorial(3)
Top -> factorial(0) returns 1
       factorial(1) returns 1 * 1
       factorial(2) returns 2 * 1
       factorial(3) returns 3 * 2
```

10.2 Balanced Parentheses

Problem: determine whether brackets close correctly.

Why it matters. The most recent opening bracket must match the next closing bracket. That is LIFO.

```
def is_balanced(text):
    pairs = {')': '(', ']': '[', '}': '{'}
    stack = []
    for ch in text:
        if ch in '([{':
            stack.append(ch)
        elif ch in pairs:
            if not stack or stack[-1] != pairs[ch]:
                return False
            stack.pop()
    return not stack
```

Time: $O(n)$ **Extra space:** $O(n)$

10.3 Next Greater Element

Problem: for each number, find the next number to its right that is greater.

Why it matters. A monotonic stack keeps unresolved candidates. When a greater value appears, it resolves recent smaller values first.

```
def next_greater(nums):
    ans = [-1] * len(nums)
    stack = []  # indices with decreasing values
    for i, value in enumerate(nums):
        while stack and nums[stack[-1]] < value:
            ans[stack.pop()] = value
```

```
        stack.append(i)
    return ans
```

10.4 Other Core Applications

Application	Why stack works	Time
Undo/redo	Last edit is undone first; redo can use another stack.	$O(1)$ per action
Browser history	Current page goes back to most recent previous page.	$O(1)$
DFS	Explore deepest recent node before alternatives.	$O(V + E)$
Backtracking	Undo latest choice first.	varies
Expression evaluation	Operators and operands are nested.	$O(n)$
Compiler parsing	Syntax scopes nest.	$O(n)$
Stock span	Monotonic stack removes weaker previous prices.	$O(n)$
Largest histogram rectangle	Stack stores increasing bars until boundary found.	$O(n)$
Rain water trapping	Stack finds left and right walls.	$O(n)$

11 Module 10: Stack Interview Patterns

Pattern	Recognition clue	Key idea
Balanced parentheses	Matching nested symbols	Push opens, pop on closes
Next greater/smaller	Nearest bigger/smaller neighbor	Monotonic stack
Daily temperatures	Wait until warmer future day	Store unresolved indices
Stock span	Consecutive previous values less/equal	Pop weaker prices
Min stack / max stack	Need minimum with pop	Auxiliary stack of minima/maxima
Queue using stacks	FIFO from LIFO tools	Input stack and output stack
Stack using queues	LIFO from FIFO tools	Rotate queue after push
Sort stack	Sort using only stacks	Temporary stack insertion
Celebrity problem	Candidate elimination	Stack or two-pointer logic
Asteroid collision	Recent moving asteroid collides first	Stack survivors
Decode string	Nested encoded substrings	Stack counts and strings
Simplify path	Last valid directory can be removed	Stack path components
Remove K digits	Remove previous larger digit	Monotonic increasing stack

11.1 Min Stack

```
class MinStack:
    def __init__(self):
        self.values = []
        self.mins = []

    def push(self, x):
        self.values.append(x)
        if not self.mins or x <= self.mins[-1]:
            self.mins.append(x)

    def pop(self):
        x = self.values.pop()
        if x == self.mins[-1]:
            self.mins.pop()
        return x

    def get_min(self):
        return self.mins[-1]
```

11.2 Implement Queue Using Two Stacks

```
class QueueUsingStacks:
    def __init__(self):
        self.inbox = []
        self.outbox = []

    def enqueue(self, x):
        self.inbox.append(x)

    def dequeue(self):
        if not self.outbox:
            while self.inbox:
                self.outbox.append(self.inbox.pop())
        if not self.outbox:
            raise IndexError("empty queue")
        return self.outbox.pop()
```

12 Module 11: Coding in Multiple Languages

12.1 C

```c
#include <stdio.h>
#define CAP 100
int stack[CAP], top = -1;
int is_empty() { return top == -1; }
int is_full() { return top == CAP - 1; }
void push(int x) { if (!is_full()) stack[++top] = x; }
int pop() { return is_empty() ? -1 : stack[top--]; }
int peek() { return is_empty() ? -1 : stack[top]; }
```

12.2 C++

```cpp
#include <vector>
#include <stdexcept>
class Stack {
    std::vector<int> data;
public:
    void push(int x) { data.push_back(x); }
    int pop() { if (data.empty()) throw std::runtime_error("empty"); int x=data.back(); data.
    pop_back(); return x; }
    int peek() const { if (data.empty()) throw std::runtime_error("empty"); return data.back
    (); }
};
```

12.3 Java

```java
import java.util.ArrayDeque;
class DemoStack {
    private ArrayDeque<Integer> st = new ArrayDeque<>();
    void push(int x) { st.push(x); }
    int pop() { return st.pop(); }
    int peek() { return st.peek(); }
    boolean isEmpty() { return st.isEmpty(); }
}
```

12.4 Python

```python
st = []
st.append(10)      # push
x = st.pop()       # pop
y = st[-1]         # peek
empty = len(st) == 0
```

12.5 JavaScript

```
const stack = [];
stack.push(10);
const x = stack.pop();
const top = stack[stack.length - 1];
```

12.6 C#

```
using System.Collections.Generic;
Stack<int> st = new Stack<int>();
st.Push(10);
int x = st.Pop();
int top = st.Peek();
```

12.7 Go

```
type Stack []int
func (s *Stack) Push(x int) { *s = append(*s, x) }
func (s *Stack) Pop() int { old := *s; x := old[len(old)-1]; *s = old[:len(old)-1]; return x
    }
```

12.8 Rust

```
let mut stack: Vec<i32> = Vec::new();
stack.push(10);
let x = stack.pop();
let top = stack.last();
```

13 Module 12: Practice Questions

The full practice ladder below is arranged in learning order. For each problem, use this method: state why the problem exists, try brute force, find the stack clue, dry run, code, analyze complexity, then list mistakes.

13.1 30 Easy

1. Implement stack using array.
2. Implement stack using linked list.
3. Reverse a string.
4. Check balanced parentheses.
5. Delete middle of stack.
6. Insert at bottom.
7. Reverse stack recursively.
8. Evaluate simple postfix.
9. Convert decimal to binary.
10. Browser back simulation.
11. Undo text editor operation.
12. Validate pushed/popped sequences.
13. Remove adjacent duplicates.
14. Baseball game scoring.
15. Make string good.
16. Backspace string compare.
17. Final prices with discount.
18. Implement min stack.
19. Implement max stack.
20. Sort a small stack.
21. Copy stack preserving order.
22. Search in stack.
23. Clear stack safely.
24. Display top to bottom.
25. Display bottom to top.
26. Count stack size recursively.
27. Parentheses depth.
28. Remove outer parentheses.
29. Simplify small path.
30. Design fixed stack.

13.2 30 Medium

1. Next greater element I.
2. Next greater element II.
3. Next smaller element.
4. Previous greater element.
5. Previous smaller element.
6. Daily temperatures.
7. Stock span.
8. Asteroid collision.
9. Decode string.
10. Simplify Unix path.
11. Remove K digits.
12. Online stock span.
13. Queue using stacks.
14. Stack using queues.
15. Infix to postfix.
16. Postfix evaluation.
17. Prefix evaluation.
18. Basic calculator I.
19. Basic calculator II.
20. Exclusive time of functions.
21. Validate serialization.
22. Score of parentheses.

23. Minimum remove to valid parentheses.
24. Remove duplicate letters.
25. Sum of subarray minimums.
26. 132 pattern.
27. Collision of robots.
28. Build array with stack operations.
29. Flatten nested list iterator.
30. DFS iterative traversal.

13.3 30 Hard

1. Largest rectangle in histogram.
2. Maximal rectangle.
3. Trapping rain water.
4. Basic calculator III.
5. Number of visible people.
6. Sum of subarray ranges.
7. Maximum min-product.
8. Remove duplicate letters optimally.
9. Smallest subsequence distinct.
10. Create maximum number.
11. Count submatrices with all ones.
12. Parse ternary expression.
13. Longest valid parentheses.
14. Expression add operators.
15. Boolean expression parser.
16. Design browser history.
17. Design persistent stack.
18. Lock-free stack concept.
19. Compiler bracket parser.
20. Iterative Tarjan-style DFS idea.
21. Monotonic stack with binary search.
22. Cartesian tree from stack.
23. Maximum width ramp.
24. Beautiful towers.
25. Robot collisions advanced.
26. Remove nodes from linked list.
27. Count valid subarrays.
28. Car fleet II.
29. Dinner plate stacks.
30. Expression conversion all forms.

13.4 Problem Template Example: Balanced Parentheses

- Why: teaches nested matching.
- Hint 1: store opening brackets.
- Hint 2: closing bracket must match most recent opening bracket.
- Hint 3: empty stack at the end means all opens closed.
- Brute force: repeatedly remove matching pairs; can be $O(n^2)$.
- Optimal: stack; $O(n)$ time and $O(n)$ space.
- Common mistake: accepting "([)]" because counts match; order matters.

14 Module 13: Advanced Topics

14.1 Multiple Stacks in One Array

Two stacks can share one array by growing from opposite ends.

```
left stack grows ->        <- right stack grows
+----+----+----+----+----+----+
| L1 | L2 |    |    | R2 | R1 |
+----+----+----+----+----+----+
```

Overflow occurs only when the two tops meet.

14.2 Dynamic and Generic Stacks

A dynamic stack resizes as needed. A generic stack stores any type while preserving type safety, such as `Stack<T>` in Java/C# or templates in C++.

14.3 Thread-Safe and Lock-Free Stacks

A thread-safe stack protects operations when multiple threads access it. A lock-free stack uses atomic compare-and-swap so at least one thread makes progress without traditional locks. These are advanced because memory ordering and ABA problems can cause subtle bugs.

14.4 Persistent, Functional, and Immutable Stacks

An immutable stack does not modify old versions. Push returns a new stack head pointing to the old stack. This is powerful in functional programming, undo systems, and versioned data.

14.5 Systems Applications

- Operating systems: kernel stacks, interrupt handling, process execution.
- Compilers: parsing, syntax checking, expression evaluation.
- Virtual machines: operand stacks in bytecode interpreters.
- Databases: query execution, transaction rollback ideas.
- Networking: protocol parsing and nested packet handling.
- Browser engines: JavaScript call stack, DOM traversal, navigation history.
- Game engines: state rollback, scene graph traversal, AI search.
- AI systems: DFS, planning, backtracking, expression trees.

15 Module 14: Interview Preparation

15.1 How Interviewers Think

Interviewers do not merely want a memorized definition. They look for recognition: can you see that a problem is nested, reversible, or nearest-neighbor based? They also watch communication, edge cases, and complexity reasoning.

15.2 Stack Interview Checklist

1. State the stack clue: LIFO, nesting, undo, nearest greater/smaller, unresolved items.
2. Define what each stack element stores: value, index, pair, state, or object.
3. Dry run on a small example.
4. Handle empty input, duplicates, and invalid operations.
5. Explain why each item is pushed and popped at most once for $O(n)$ monotonic stack solutions.
6. Code cleanly with meaningful variable names.

15.3 Common Follow-Ups

- Can you return minimum in $O(1)$?
- Can you avoid extra space?
- Can this be made thread-safe?
- What happens on overflow or underflow?
- Can recursion be converted to an explicit stack?

16 Module 15: Revision

16.1 Mind Map

```
Stack
|-- Rule: LIFO
|-- Core operations: push, pop, peek
|-- Implementations
|   |-- Array: contiguous, fast, capacity/resizing
|   |-- Linked list: dynamic, pointer overhead
|-- Applications
|   |-- Calls, recursion, undo, browser history
|   |-- Parsing, parentheses, expressions
|   |-- DFS, backtracking
|   |-- Monotonic stack patterns
|-- Risks: overflow, underflow, wrong order
```

16.2 One-Page Cheat Sheet

Idea	Memory hook
LIFO	Last in, first out
Top	Only accessible end
Push	Add to top
Pop	Remove from top
Peek	Read top
Underflow	Pop/peek empty stack
Overflow	Push full fixed stack
Array stack	top index
Linked stack	head pointer
Monotonic stack	Stack kept increasing/decreasing

16.3 Decision Tree

```
Need most recent item first? -> Stack
Need oldest item first?      -> Queue
Need random index access?    -> Array/list
Need nearest greater/smaller?-> Monotonic stack
Need nested matching?        -> Stack
Need priority order?         -> Heap
```

16.4 Flashcards

1. Q: What does LIFO mean? A: Last inserted item is removed first.
2. Q: Why is push $O(1)$? A: it touches only the top.
3. Q: Why is search $O(n)$? A: many items may need inspection.

4. Q: What is stack underflow? A: removing or reading from an empty stack.
5. Q: What is a monotonic stack? A: a stack maintained in increasing or decreasing order to solve nearest-neighbor problems.

16.5 Final Mastery Quiz

1. Explain stack to a five-year-old using plates.
2. Explain stack to an interviewer using LIFO and operations.
3. Implement array stack and linked stack.
4. Derive time complexity of push, pop, peek, display, and search.
5. Solve balanced parentheses and next greater element.
6. Explain why recursion uses a call stack.
7. Compare array and linked-list stacks for production use.

Final Summary

A stack is one of the most important data structures because it captures a universal rule: the most recent unfinished thing is handled first. This rule appears in memory management, function execution, undo systems, parsing, recursion, DFS, backtracking, expression evaluation, and many interview patterns. Mastering stacks means mastering not just push and pop, but also the ability to recognize LIFO-shaped problems.